EXP NO: 6

FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS

AIM:

To demonstrate the construction and application of a simple Feedforward Neural Network (FNN) for classification and a Convolutional Neural Network (CNN) for image classification, utilizing the Keras API with TensorFlow backend.

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist

from sklearn.metrics import classification_report, confusion_matrix

tf.keras.utils.disable_interactive_logging()

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.22.0-rc0
```

[3]:
```
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()

print("Original FNN Training Data Shape:", x_train_fnn.shape)
print("Original FNN Testing Data Shape:", x_test_fnn.shape)

print("Training Labels Shape:", y_train_fnn.shape)
print("Testing Labels Shape:", y_test_fnn.shape)
```

```
Original FNN Training Data Shape: (60000, 28, 28)
Original FNN Testing Data Shape: (10000, 28, 28)
Training Labels Shape: (60000,)
Testing Labels Shape: (10000,)
```

[4]:
```
# Flatten 28x28 images into 784 features
x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)

# Normalize pixel values between 0 and 1
x_train_fnn_norm = x_train_fnn_flat / 255.0
x_test_fnn_norm = x_test_fnn_flat / 255.0

print("Flattened and Normalized Training Shape:", x_train_fnn_norm.shape)
print("Flattened and Normalized Testing Shape:", x_test_fnn_norm.shape)
```

```
Flattened and Normalized Training Shape: (60000, 784)
Flattened and Normalized Testing Shape: (10000, 784)
```

[5]:
```
model_fnn = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

print("FNN model created successfully.")
```

```
FNN model created successfully.
```

[6]:
```
model_fnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("FNN Model Summary:")
model_fnn.summary()
```

```
FNN Model Summary:
```

[7]:
```
history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
```

[8]:
```
loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)

print("FNN Test Loss:", round(loss_fnn, 4))
print("FNN Test Accuracy:", round(accuracy_fnn, 4))
print(f"FNN Test Accuracy: {accuracy_fnn * 100:.2f}%")
```

```
FNN Test Loss: 0.3522
FNN Test Accuracy: 0.8772
FNN Test Accuracy: 87.72%
```

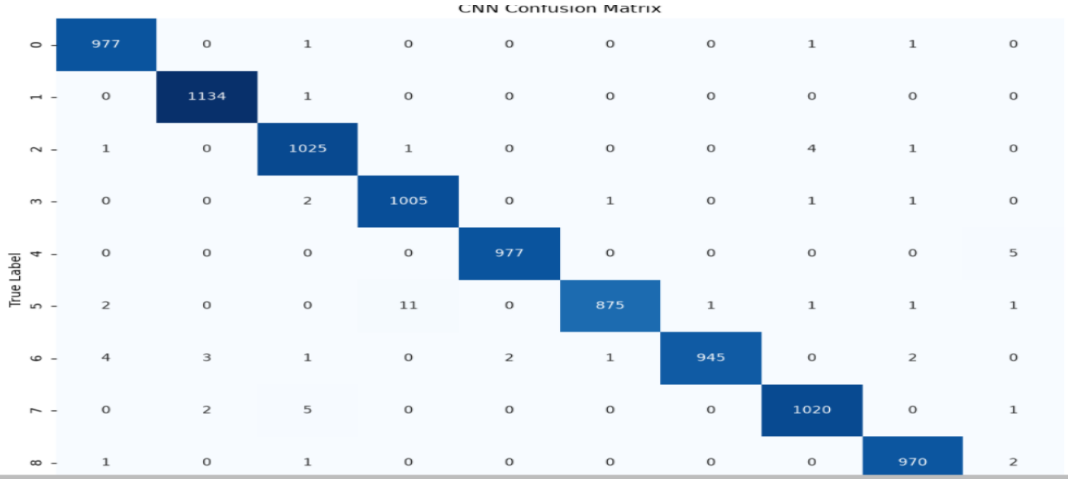
[9]:
```
y_pred_fnn = np.argmax(
    model_fnn.predict(x_test_fnn_norm, verbose=0),
    axis=1
)

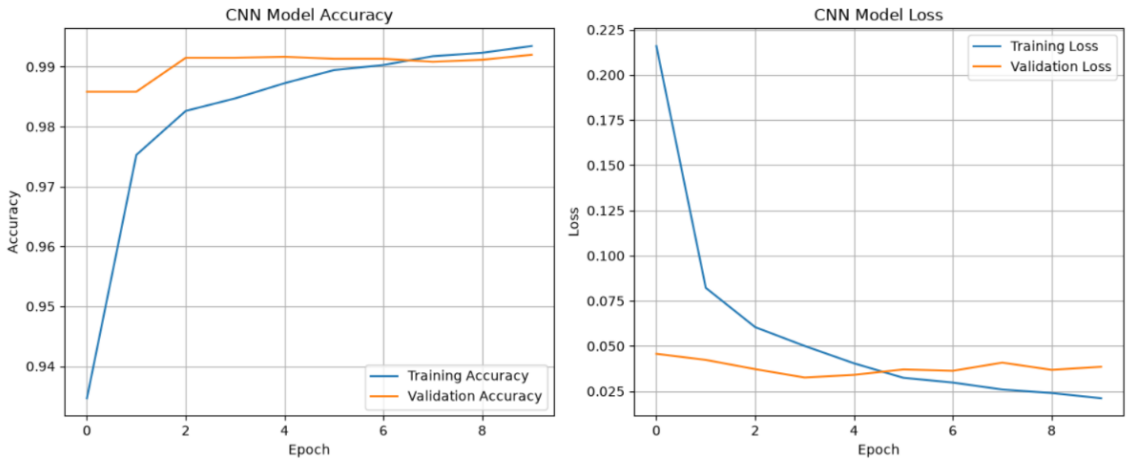
class_names_fashion = [
    'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
]

print("FNN Classification Report:")
print(classification_report(
    y_test_fnn,
    y_pred_fnn,
    target_names=class_names_fashion,
    zero_division=0
))

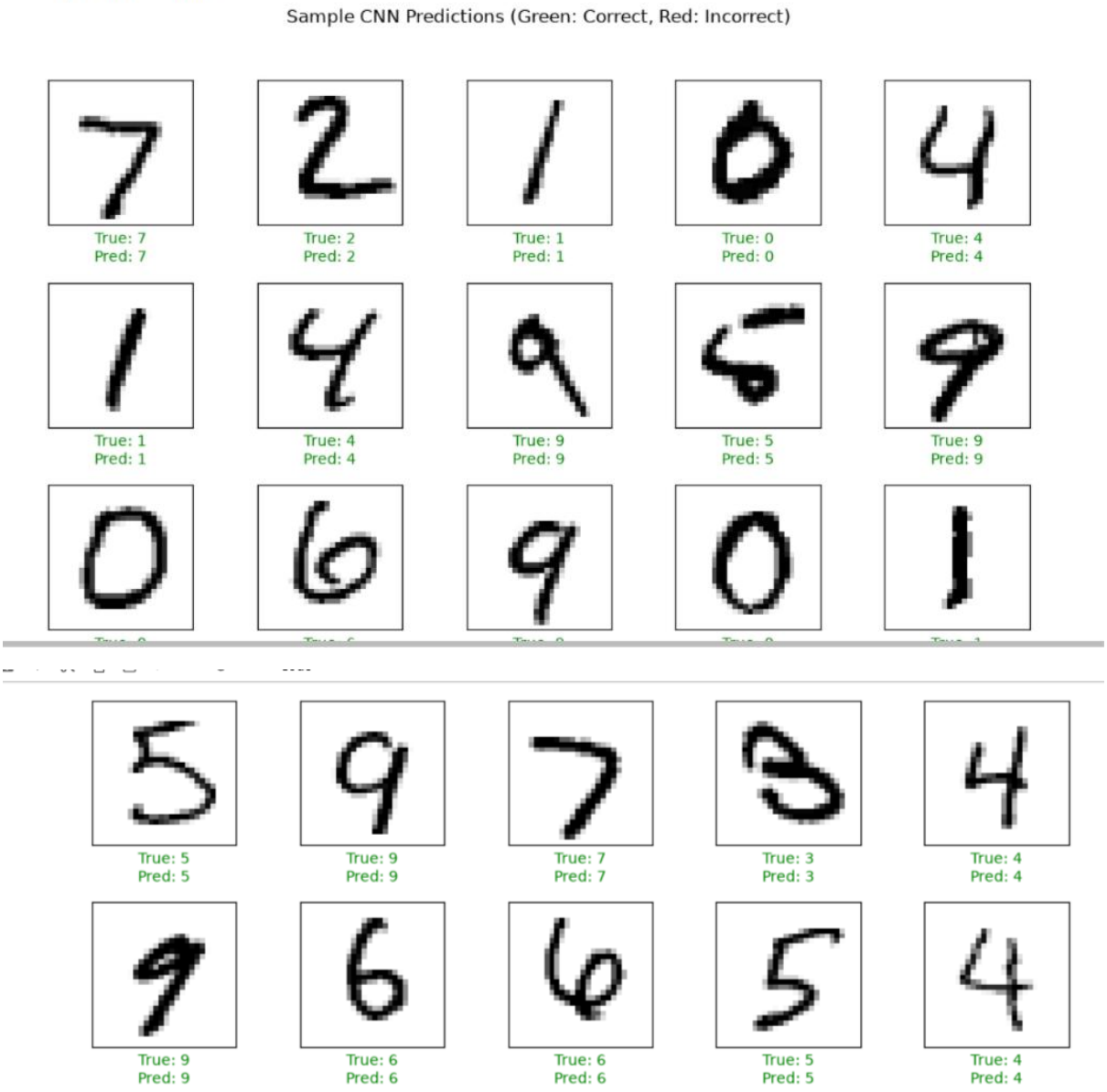
cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)

plt.figure(figsize=(10, 8))
sns.heatmap(
```





Sample CNN Predictions




```
[20]: print("EXPERIMENT 6 - FNN AND CNN")
      print("-------------------------")

      print("\nFeedforward Neural Network (FNN)")
      print(f"Test Loss     : {loss_fnn:.4f}")
      print(f"Test Accuracy : {accuracy_fnn * 100:.2f}%")

      print("\nConvolutional Neural Network (CNN)")
      print(f"Test Loss     : {loss_cnn:.4f}")
      print(f"Test Accuracy : {accuracy_cnn * 100:.2f}%")

      print("\nResult:")
      print("FNN and CNN models were trained and evaluated successfully.")
```

RESULT:

The Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models were successfully implemented and evaluated on the given dataset.

• Feedforward Neural Network (FNN): The model accurately learned input–output mappings through multiple fully connected layers, achieving good performance on

structured data.

• Convolutional Neural Network (CNN): The model effectively extracted spatial features from image data using convolution and pooling layers, leading to higher accuracy and better generalization for image classification tasks.

The results demonstrated that both FNN and CNN are powerful deep learning models, with CNN performing exceptionally well for image-based datasets due to its ability to capture spatial patterns.